Ders 2 Çalışma Özeti

Ders 2 Çalışma Özeti

Genel Konular

- Sınıflandırma ve regresyon ayrımı
  - Şimdiye kadar görülen eğitmenli (supervised) yöntemlerin hepsi sınıflandırmaydı: giriş ayrık ya da sürekli olabilir, çıkış her zaman ayrıktır, yani bir sınıftır (ekrandan, 00:10:00).
  - Regresyonda giriş yine ayrık ya da sürekli olabilir, ama çıkış süreklidir. Amaç bir sınıf değil, bir değer tahmin etmektir: $f : X \to Y$.
  - Derste verilen örnekler: bir filmin alacağı ortalama puan, bir Instagram hesabının altı ay sonraki takipçi sayısı, özellikleri verilen bir evin fiyatı, hava ve toprak koşullarına göre mısır hasadı, yıllara göre satış fiyatı.
- Doğrusal regresyon modeli
  - Giriş vektörü özelliklerden oluşur: $X = (x_1, \dots, x_d)$ (ekrandan, 00:13:36).
  - Model: $f(x) = w_0 x_0 + w_1 x_1 + \cdots + w_d x_d = \sum_{i=0}^{d} x_i w_i$.
  - w_0 bias (eşik) ağırlığıdır. Ayrı bir eşik tutmak yerine $x_0 = 1$ alınır ve w_0 öteki ağırlıklar gibi öğrenilir.
  - Girişler ve istenen çıkışlar değişmez. Öğrenme, hatayı en küçük yapan ağırlıkları bulmaktır. Bu mantık perceptron ile aynıdır ve yöntem ayırt edici (discriminative) bir yöntemdir.
- Kayıp (loss) fonksiyonları (ekrandan, 00:20:00)
  - Mean Absolute Error (L1 loss): $MAE = \frac{1}{n} \sum_{i=1}^{n} |y_i - y_i'|$.
  - Mean Squared Error (L2 loss): $MSE = \frac{1}{n} \sum_{i=1}^{n} (y_i - y_i')^2$. En çok kullanılan hatadır.
  - Root MSE: MSE'nin karekökü.
  - Huber loss: hata küçükse karesel, büyükse doğrusal davranır; aykırı değerlerin (outlier) etkisini sınırlar.
  - Gösterim: y_i istenen (etiketli) çıkış, y_i' modelin hesapladığı çıkıştır.
- Gradyan inişi ile ağırlık güncelleme (ekrandan, 00:23:08)
  - Hata: $J_n(w) = \frac{1}{n} \sum_{i=1}^{n} (y_i - w^T x_i)^2$.
  - Türev vektörü sıfıra eşitlenir: $\nabla_w J_n(w) = -\frac{2}{n} \sum_{i=1}^{n} (y_i - w^T x_i)\, x_i = 0$.
  - Tek örnek için $Error_i(w) = \frac{1}{2}(y_i - y_i')^2$ alınırsa güncelleme miktarı: $\Delta w_j = (y_i - y_i')\, x_{i,j}$.
  - Öğrenme oranıyla güncelleme: $w_j \leftarrow w_j + \alpha (y_i - y_i')\, x_{i,j}$.
- Online öğrenme algoritması
  - Ağırlıklar (w_0 dahil) sıfıra yakın küçük rastgele değerlerle başlatılır; hoca -0.5 ile $+0.5$ arasını söyledi.
  - Öğrenme oranı α 0 ile 1 arasında seçilir.
  - Bir örnek seçilir, gerçek (hesaplanan) çıkış bulunur, ağırlıklar güncellenir. Hata belirli bir ϵ değerinin altına inene kadar sürer.
- Güncelleme türleri (ekrandan, 00:32:35)
  - Online öğrenme: her örnekten sonra güncelleme.
  - Batch gradient descent: bütün örnekler gösterildikten sonra tek güncelleme.
  - Mini-batch gradient descent: belli sayıda örnekte bir güncelleme (örneğin 50 örnekte bir). Genelde tercih edilen budur; derin öğrenmede de kullanılır.
- Giriş normalizasyonu (ekrandan, 01:00:33)
  - Min-max normalizasyon: $x_i' = \frac{x_i - x_{min}}{x_{max} - x_{min}}$.

  - Standardizasyon: $x'_i = \frac{x_i - \bar{x}}{\sigma}$. Verinin normal (Gauss) dağıldığı biliniyorsa kullanılır.
- Genelleme: underfitting ve overfitting (ekrandan, 01:02:45)
  - Underfitting: model örneklerden çok uzakta kalır, öğrenme tamamlanmamıştır.
  - Overfitting: model eğitim verisini çok hassas öğrenir, genelleme yapamaz, test başarısı düşer.
- Düzenlileştirme (regularization) (ekrandan, 01:10:00)
  - Düzenlileştirilmiş maliyet = MSE + düzenlileştirme terimi.
  - Ridge regresyon (L2 norm, varsayılan): $MSE + \alpha \cdot \frac{1}{2} \sum w_i^2$.
  - Lasso regresyon (L1 norm): $MSE + \alpha \sum |w_i|$. Önemsiz özelliklerin ağırlıklarını sıfıra götürür, böylece özellik seçimi de yapar.
  - Elastic Net: $MSE + \alpha \frac{1-r}{2} \sum w_i^2 + \alpha\, r \sum |w_i|$. $r = 1$ ise Lasso, $r = 0$ ise Ridge olur.
- Lojistik regresyon (ekrandan, 01:17:57)
  - Adında regresyon geçse de bir sınıflandırma yöntemidir (ikili sınıflandırma). Kesin sınıf yerine 0 ile 1 arasında bir skor üretir.
  - $z = \sum_{i=0}^{n} x_i w_i$ toplamı lojistik fonksiyondan (sigmoid) geçirilir: $g(z) = \frac{1}{1+e^{-z}}$.
  - Karar: çıkış 0.5'ten ya da seçilen eşik θ'dan küçükse 0, değilse 1.
  - Ağırlık güncelleme: $w_j(t+1) = w_j(t) + \alpha_i \left[y_i - \frac{1}{1+e^{-\sum_j x_{i,j} w_j}} \right] x_{i,j}$ (ekrandan, 01:25:36).

Hocanın Özellikle Vurguladığı Kısımlar

- Öğrenme mantığı bütün bu yöntemlerde aynıdır
  - Giriş ve çıkış değiştirilemez; ağırlıklar rastgele başlatılır ve hatayı azaltacak yönde yinelemeli olarak güncellenir. Perceptron, doğrusal regresyon ve lojistik regresyon bu açıdan aynıdır.
  - Güncellemenin yönü: $y_i - y'_i > 0$ ise hesaplanan çıkış istenenden küçüktür, ağırlık artırılır; $y_i - y'_i < 0$ ise hesaplanan çıkış büyüktür, ağırlık azaltılır.
- Öğrenme oranının seçimi
  - Büyük α hızlı ilerler ama global minimumu atlayıp geçebilir; geçerse oraya bir daha dönülemeyebilir.
  - Başta büyük, sonra küçük α alınabilir (örneğin ilk iterasyonlarda 0.8, sonra 0.2), ya da baştan küçük bir değer seçilir.
- MSE ve aykırı değerler
  - Kare alma büyük farkları daha da büyütür; aykırı değer varsa hata çok büyür. Huber loss bu etkiyi sınırlamak için tasarlanmıştır.
- Normalizasyonu unutmamak
  - Yapay zekâda yüzde yüz başarı olmadığı için yanlış yapılan bir ön işlem fark edilmeyebilir: model yine çalışır, ama başarısı düşük kalır.
  - Örnek: binler mertebesindeki bir özellikle 1-5 arasındaki bir puan özelliği normalize edilmeden kullanılırsa etkileri anlamsız biçimde farklı olur. Fiyatın 1 milyondan 999 bine inmesi küçük bir farktır, puanın 3'ten 6'ya çıkması büyük bir farktır.
- Overfitting sık rastlanan bir sorundur
  - Underfitting daha çok tecrübesizlikten, öğrenmeyi erken kesmekten olur. Overfitting ise çok sık başa gelir; düzenlileştirme bunu önlemenin yollarından biridir.
- Lojistik regresyonun skor üretmesi
  - Bankacılık gibi alanlarda (örneğin dolandırıcılık tespiti) kesin "evet/hayır" yerine skor tercih ediliyor. Hoca kendi çalışmalarında kesin sınıflandırmayı tercih ettiğini, çünkü hangi skorun yeterli sayılacağının yine tecrübe gerektirdiğini söyledi.

Kısa Tekrar Notları

- Regresyon: çıkış süreklidir; sınıflandırma: çıkış bir sınıftır.
- Doğrusal model: $y' = \sum_{i=0}^{d} w_i x_i$, $x_0 = 1$ bias girişidir.
- Kayıplar: MAE (L1), MSE (L2), RMSE, Huber.
- Güncelleme: $w_j \leftarrow w_j + \alpha(y_i - y'_i)x_{i,j}$.
- Online, batch ve mini-batch; pratikte mini-batch.
- Ağırlıklar $[-0.5, 0.5]$ aralığında küçük rastgele değerlerle, α 0 ile 1 arasında başlatılır.

- Normalizasyon: min-max ya da standardizasyon (z-skoru).
- Ridge = L2, Lasso = L1 (özellik eler), Elastic Net = ikisinin karışımı (r ile ayarlanır).
- Lojistik regresyon = sigmoid çıkışlı ikili sınıflandırıcı; çıkış $[0, 1]$ aralığında skor, eşik 0.5 ya da θ.
- Lojistik regresyonda güncelleme aynıdır; yalnız y' sigmoid ile hesaplanır.

Detaylı Açıklamalar

- Regresyon problemi
  - Sınıflandırmada bir fotoğraf için "bu kişi şudur" ya da bir e-posta için "spam/spam değil" deriz; her etiket bir sınıftır. Regresyonda ise "bu filmin puanı 3.5 olur" ya da "bu evin fiyatı şu kadardır" deriz; sınıf yoktur, sürekli bir değer vardır.
  - Hoca tek özellikli bir örnek çizdi: yatay eksende yıl (2000, 2002, 2004...), düşey eksende fiyat (ekrandan, 00:10:00). Eğitim örneklerine bir doğru oturtulur; her örneğin bu doğruya olan dikey uzaklığı hatadır. Amaç, bütün n örnek için hatası en düşük eşlemeyi veren f fonksiyonunu bulmaktır.
- Model ve bias
  - Ekrandaki çizimde $x_1, \dots, x_d$ girişleri $w_1, \dots, w_d$ ağırlıklarıyla bir toplam düğümüne girer; ayrıca $x_0 = 1$ girişi w_0 (bias, θ) ağırlığıyla bağlanır. Çıkış $f(x, w) = \sum_{i=0}^{d} x_i w_i$ olur (ekrandan, 00:13:36).
  - Örneğin bir film için istenen ortalama puan 4, modelin hesapladığı 3.5 ise aradaki fark hatadır. Öğrenme bu hatayı küçültecek biçimde ağırlıkları yinelemeli olarak güncellemektir; ağırlıklar tek seferde bulunamaz.
- Kayıp fonksiyonları
  - MAE farkların mutlak değerlerinin ortalamasıdır. MSE farkların karelerinin ortalamasıdır ve hatayı daha belirgin hâle getirir; ancak aykırı değer varsa hata çok büyür ("outlier varsa hata büyüyor", ekrandan, 00:20:00).
  - Huber loss ekrandaki biçimiyle (ekrandan, 00:20:00):

$$L(y_i, y_i') = \begin{cases} \frac{1}{2}(y_i - y_i')^2 & |y_i - y_i'| \le \delta \\ \delta\left(|y_i - y_i'| - \frac{1}{2}\delta\right) & \text{aksi hâlde} \end{cases}$$

  - Hata eşikten küçükse sıradan bir hatadır ve karesi alınır. Büyükse doğrusal ceza verilir; böylece tek bir aykırı değer hatayı aşırı büyütüp öğrenmeyi bozamaz.
  - Ders sonrası not: Hoca Root MSE'yi tahtaya $\frac{1}{n}\sqrt{\sum(y_i - y_i')^2}$ diye yazdı; doğrusu $RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - y_i')^2}$ biçimindedir, yani $\frac{1}{n}$ karekökün içindedir.
- Gradyan inişi
  - Hata yüzeyinde ağırlıklar değiştikçe hata azalır; amaç global minimuma ulaşmaktır, yolda yerel minimumlar olabilir. Hoca teorik türetmeyi atlayıp yalnız sonucu hatırlattı: hatanın türevi alınıp sıfıra eşitlenir ve ağırlıklar buna göre güncellenir.
  - Güncelleme kuralı $\Delta w_j = (y_i - y_i')x_{i,j}$ perceptron kuralıyla aynıdır.
  - Hoca tahtada bir doğrunun iterasyonlar boyunca adım adım örneklere yaklaştığını çizdi (ekrandan, 00:32:35): ilk ağırlıklarla elde edilen doğru örneklerden uzaktır, her güncellemede biraz daha yaklaşır.
- Güncelleme sıklığı
  - Her örnekten sonra güncellemek (online) yavaştır. Bin örneğin hepsini gösterip tek güncelleme yapmak (batch) da etkili değildir. Bu yüzden genelde belli sayıda örnekte bir güncellenir (mini-batch).
- Normalizasyon
  - Min-max her özelliği kendi en küçük ve en büyük değerine göre aynı aralığa getirir. Standardizasyon ortalamayı çıkarıp standart sapmaya böler. Hoca min-max'ın daha çok kullanıldığını, farklı normalizasyonların denenip görselleştirilerek seçilebileceğini söyledi.
- Düzenlileştirme
  - Amaç modelin daha iyi genellemesini sağlamak, overfitting olasılığını azaltmaktır. Ridge bütün ağırlıkların karesini maliyete ekler ve ağırlıkların dengesini korur. Lasso bazı özellikler daha önemliyse kullanılır; önemsiz özelliklerin ağırlığı sıfır olur ve o özellik elenir.

  - Elastic Net'te $r = 1$ alınırsa $(1 - r)$ terimi sıfır olur ve geriye Lasso kalır; $r = 0$ alınırsa ikinci terim sıfır olur ve geriye Ridge kalır.
  - Ders sonrası not: Hoca Lasso ve Elastic Net'in L1 terimini tahtaya $\sum |w_i|^2$ diye yazdı (ekrandan, 01:10:00), ama aynı yerde bunu "L1 norm" diye adlandırdı. L1 terimi $\sum |w_i|$ biçimindedir; kare alınırsa terim Ridge'deki L2 terimine dönüşür.
  - Ders sonrası not: Hoca α'nın 0 ile 1 arasında değiştiğini söyledi. Büyük olasılıkla bu derste kullanılan pratik aralıktır; genel olarak $\alpha \geq 0$ herhangi bir değer olabilir ve 1'den büyük de seçilebilir.
- Lojistik regresyon
  - Karar sınırı $w^T x = 0$ doğrusudur; iki sınıfı en iyi ayıran sınır ağırlıklarla belirlenir (ekrandan, 01:17:57).
  - Tahtada iki geçiş biçimi karşılaştırıldı: hard switching (basamak) ve logistic (smooth) switching (sigmoid). Sigmoid çok katmanlı perceptronda kullanılan fonksiyondur.
  - Ders sonrası not: Tahtadaki karar kuralı "$P(y = 0 \mid x) < 0.5$ ise 0" diye yazılmıştı (ekrandan, 01:20:02). Sigmoid çıkışı $P(y = 1 \mid x)$ olarak yorumlanır; doğru kural "$f(x) = P(y = 1 \mid x) < 0.5$ (ya da $< \theta$) ise 0, değilse 1" biçimindedir. Hocanın sözlü anlatımı da bu doğru kurala uyuyor.
  - Örnek (ekrandan, 01:22:00): duygu analizi için dört özellik: x_1 tweetteki olumlu kelime sayısı, x_2 olumsuz kelime sayısı, x_3 zamir sayısı (I, you, me…), x_4 toplam kelime sayısı. $(5, 1, 4, 6)$ özellikli bir örnek 1 (olumlu) olarak etiketlenir. Bilinmeyen bir örnekte çıkış 1'e yakınsa tweet olumlu, 0'a yakınsa olumsuz sayılır.
  - Sayısal örnek (ekrandan, 01:28:18): $x_0 = 1$, $x_1 = 1$, $x_2 = 4$; $w_0 = 0.2$, $w_1 = 0.1$, $w_2 = 0.3$. $z = 0.2 + 0.1 + 0.3 \cdot 4 = 1.5$, $y' = \frac{1}{1+e^{-1.5}} \approx 0.818$. w_1 güncellenirken $w_1 \leftarrow w_1 + \alpha(y_i - y_i')x_1$ kullanılır. İstenen çıkış y_i derste verilmedi, bu yüzden güncellemenin sonucu hesaplanmadı.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.